Practical No. 12 : Implement and evaluate a neural network to classify data into multiple classes based on predictor variables.

Submitted By: Shantanu Yadav
Roll No.: 75

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
```

In [2]:
```
from sklearn.datasets import load_iris

iris = load_iris()

df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target
df['species_name'] = df['species'].map(dict(enumerate(iris.target_names)))

df.head()
```

Out[2]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species	species_name
0	5.1	3.5	1.4	0.2	0	setosa
1	4.9	3.0	1.4	0.2	0	setosa
2	4.7	3.2	1.3	0.2	0	setosa
3	4.6	3.1	1.5	0.2	0	setosa
4	5.0	3.6	1.4	0.2	0	setosa

In [3]:
```
print('Shape of dataset:', df.shape)
print('\nClass distribution:')
print(df['species_name'].value_counts())

df.describe()
```

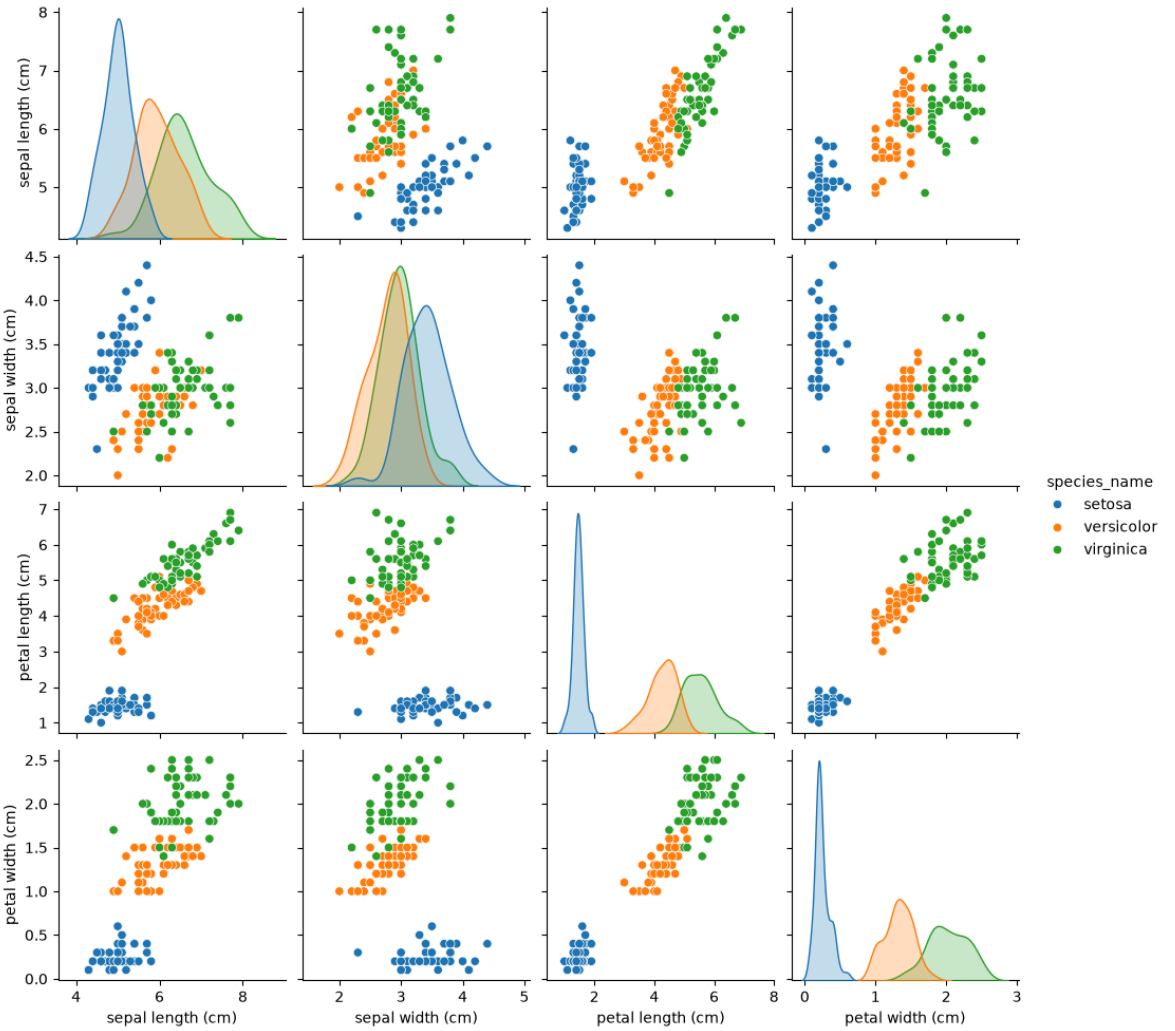
```
Shape of dataset: (150, 6)

Class distribution:
species_name
setosa        50
versicolor    50
virginica     50
Name: count, dtype: int64
```

Out[3]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333	1.000000
std	0.828066	0.435866	1.765298	0.762238	0.819232
min	4.300000	2.000000	1.000000	0.100000	0.000000
25%	5.100000	2.800000	1.600000	0.300000	0.000000
50%	5.800000	3.000000	4.350000	1.300000	1.000000
75%	6.400000	3.300000	5.100000	1.800000	2.000000
max	7.900000	4.400000	6.900000	2.500000	2.000000

In [4]:
```
sns.pairplot(df, hue='species_name', vars=iris.feature_names)
plt.show()
```



In [5]:
```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X = df[iris.feature_names]
y = df['species']
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print('Training samples:', X_train.shape[0])
print('Testing samples:', X_test.shape[0])
```

```
Training samples: 120
Testing samples: 30
```

In [6]:
```
from sklearn.neural_network import MLPClassifier

model = MLPClassifier(
    hidden_layer_sizes=(16, 8),
    activation='relu',
    solver='adam',
    max_iter=1000,
    random_state=42
)

model.fit(X_train_scaled, y_train)

print('Neural network model trained successfully.')
print('Number of iterations:', model.n_iter_)
```

```
Neural network model trained successfully.
Number of iterations: 585
```

In [7]:
```
from sklearn.metrics import accuracy_score, classification_report, confus

y_pred = model.predict(X_test_scaled)

accuracy = accuracy_score(y_test, y_pred)
print('Accuracy:', round(accuracy, 4))
print('\nClassification Report:')
print(classification_report(y_test, y_pred, target_names=iris.target_name
```

```
Accuracy: 0.9667

Classification Report:
              precision    recall  f1-score   support

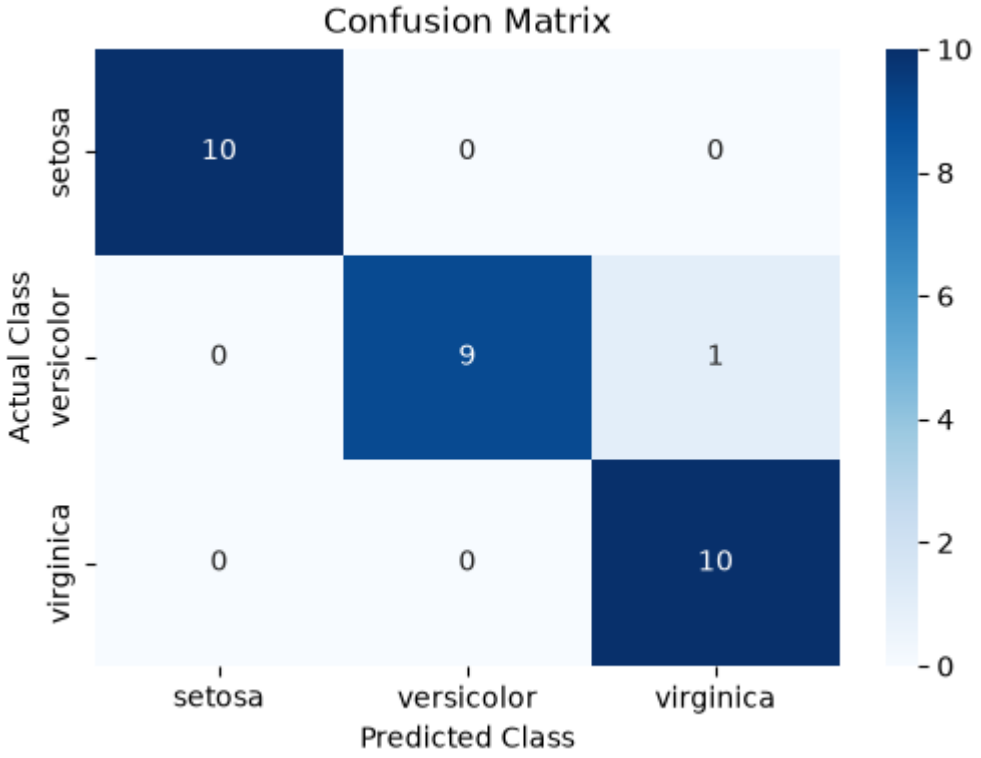
      setosa       1.00      1.00      1.00        10
  versicolor       1.00      0.90      0.95        10
   virginica       0.91      1.00      0.95        10

    accuracy                           0.97        30
   macro avg       0.97      0.97      0.97        30
weighted avg       0.97      0.97      0.97        30
```

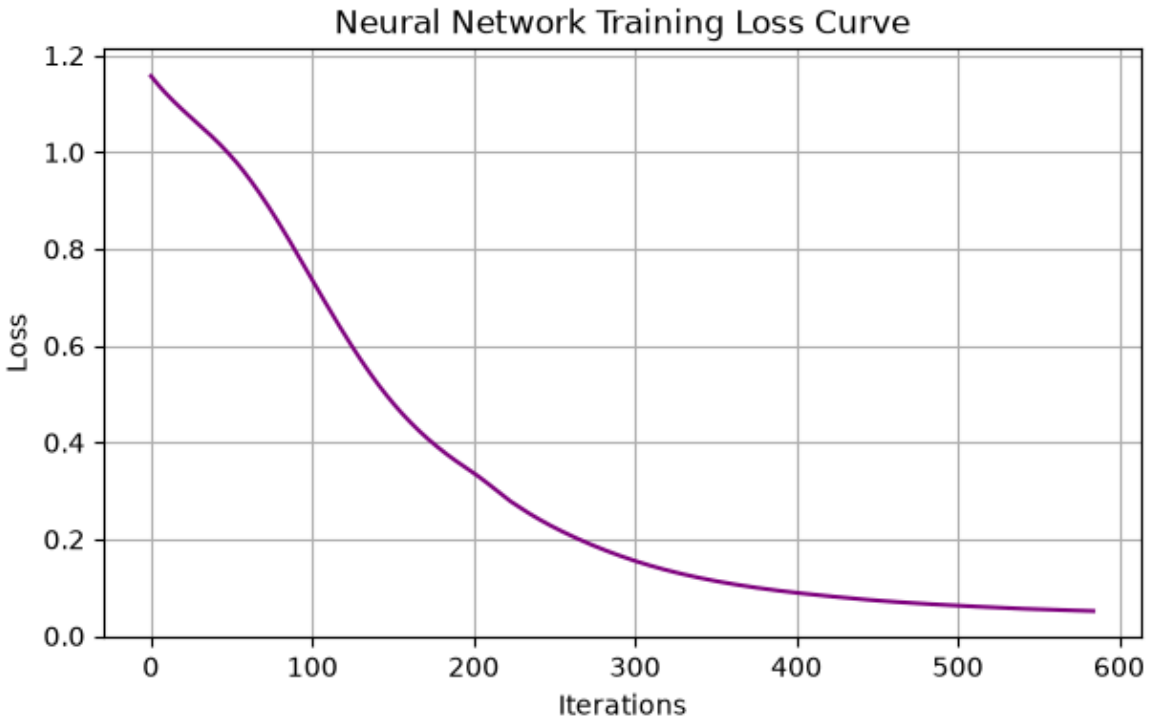
In [8]:
```
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6, 4))
sns.heatmap(
    cm,
    annot=True,
```

```
    fmt='d',
    cmap='Blues',
    xticklabels=iris.target_names,
    yticklabels=iris.target_names
)
plt.xlabel('Predicted Class')
plt.ylabel('Actual Class')
plt.title('Confusion Matrix')
plt.show()
```



In [9]:
```
plt.figure(figsize=(7, 4))
plt.plot(model.loss_curve_, color='purple')
plt.xlabel('Iterations')
plt.ylabel('Loss')
plt.title('Neural Network Training Loss Curve')
plt.grid(True)
plt.show()
```



In [10]:
```
prediction_table = pd.DataFrame({
    'Actual Class': [iris.target_names[i] for i in y_test.values],
    'Predicted Class': [iris.target_names[i] for i in y_pred]
})

prediction_table.head(10)
```

Out[10]:

	Actual Class	Predicted Class
0	setosa	setosa
1	virginica	virginica
2	versicolor	versicolor
3	versicolor	versicolor
4	setosa	setosa
5	versicolor	versicolor
6	setosa	setosa
7	setosa	setosa
8	virginica	virginica
9	versicolor	versicolor